



# STRINGS

MARVIN C SANTOS

INSTRUCTOR

# STRINGS

- Is a variable contains a collection of characters surrounded by “*double quotes*”
- Is a collection of character

S

**char**

S

W

I

M

**string**

# STRINGS - SYNTAX

```
char thisLetter = S;  
string thisWord = "Swim";
```

S

**char**

S

W

I

M

**string**

# STRINGS

In C# string is an OBJECT.

It contains properties and methods

Find the length of the string “Swim”

*Total char = 4*

# VALUE TYPE AND REFERENCE TYPE

Value Type = Data types that holds a data value within its own memory.

= Directly contain value.

*int, decimal, char, double, float, long etc.*

Reference Type = Doesn't store its value directly.

= It contains a pointer to another memory location that holds the data.

*string, class*

# STRINGS

- A data type used to represent text as a sequence of characters
- One of the most commonly used data types in C#
  - Declaring and Initializing
  - Concatenating
  - Interpolation
  - String Methods
  - String Comparison
  - String Formatting
  - Null vs. Empty Strings

# DECLARING AND INITIALIZING STRINGS

- You can declare a string variable and initialize it

```
string str1 = "Hello, World!";  
string str2 = "The quick brown fox...";  
string str3 = null;  
string str4 = "";
```

# CONCATENATING STRINGS

- You can concatenate strings using the + operator

```
string firstName = "Juan";  
string lastName = "De la Cruz";  
string fullName = firstName + " " + lastName;
```

# STRING INTERPOLATION

- Using the '\$' symbol before a string, allowing us to embed expression directly in a string

```
int age = 30;  
string message = $"My age is {age}";  
Console.WriteLine(message);
```

# STRING METHODS

- These methods help you manipulate and analyze string data.
- Provides various methods for working with strings, such as Length, Substring, ToUpper, ToLower, IndexOf, Replace, and many more.

```
string text = "Hello, World!";  
int length = text.Length;  
string sub = text.Substring(7);  
string upper = text.ToUpper();  
string replaced = text.Replace("Hello", "Hi");
```

# STRING COMPARISON

- Compare strings using methods like Equals, Compare, and CompareOrdinal for various types of string

```
string fruit1 = "apple";  
string fruit2 = "Apple";  
bool areEqual = str1.Equals(str2, StringComparison.OrdinalIgnoreCase);  
Console.WriteLine(areEqual);
```

# STRING FORMATTING

- using methods like 'string.Format' or interpolated strings for more complex formatting tasks.

```
int num = 42;  
string formatted = string.Format("The answer is {0}", num);
```

# NULL VS. EMPTY STRINGS

- Null string reference means the variable does not point to any string object, while an empty string is a valid string with no characters.

```
string nullString = null;  
string emptyString = "";
```

## STRINGS

- Access individual characters in a string using indexing

```
string middleName = "Villanueva";  
char middleInitial = middleName[0];  
Console.WriteLine(middleInitial);
```

```
string sentence1 = "The quick brown fox jumps over the lazy dog";  
char randomChar = sentence1[8];  
Console.WriteLine(randomChar);
```

## IN SUMMARY...

*“Understanding how to work with strings effectively is crucial in many C# applications, as strings are widely used for text processing, user input, and communication with external systems”*



THANK YOU...